RICE UNIVERSITY

Serialization, RPC, and Datacenter Networks

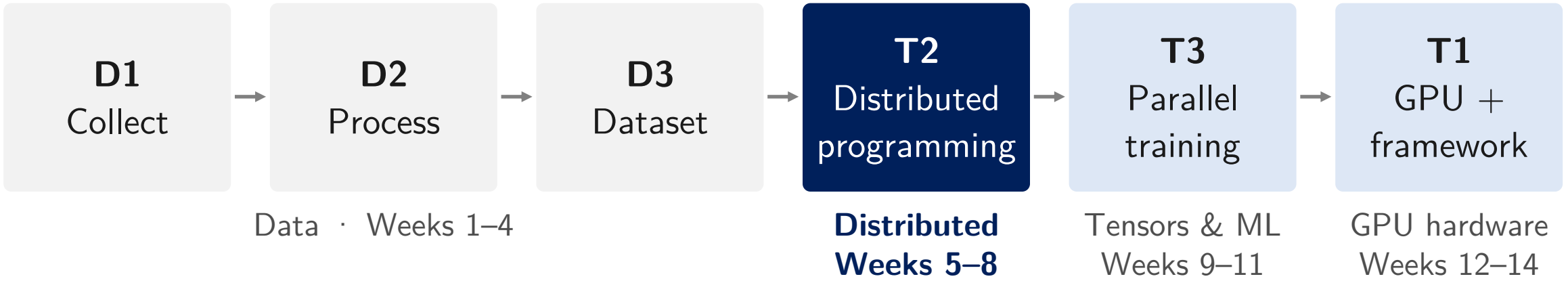
From objects to bytes, from local calls to remote ones, and the networks that carry them

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 6, Lecture 2

Where we are in the pipeline



In our pipeline

Today: objects → bytes, local calls → remote calls, and the networks of a datacenter. **D3 checkpoint this week.**

Last time (Week 6 L1): sockets and $\alpha + \beta n$. Next: message passing with mpi4py (Week 7 L1).

Why do we need serialization?

The job: send our MLP's gradients to another machine
34,338 floats; a socket only takes **bytes**

How many bytes go over the network?

Raw 4-byte floats
137 KB

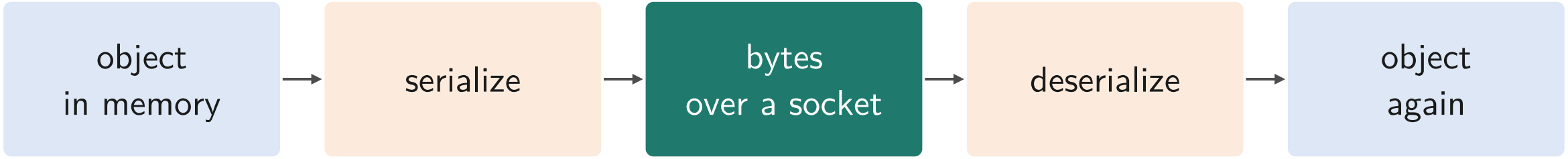
XML-RPC (Python's built-in)
1.77 MB: 13× more

Key idea

How you turn objects into bytes can cost **more than the network** itself.

Source: our laptop, made for these slides: Apple M1 Pro, macOS, Python 3.14, NumPy 2.5, loopback; median of 5 runs.

What serialization and RPC do



serialization: object $\rightarrow$ bytes that rebuild it anywhere

RPC: result = f(x) runs on **another machine**
the **stub** serializes f and x, sends them, waits, and deserializes the result

Key idea

Serialization turns objects into bytes; **RPC** turns a local call into two messages.

Where they show up

Where	What it serializes or calls	In this course
multiprocessing, mpi4py	pickle of Python objects	Week 5; Week 7
gRPC, torch.distributed.rpc	remote calls between services	mention only
Python xmlrpc	remote calls as XML text	today's build

Large training clusters add **InfiniBand** or **RDMA over Ethernet** (Part 3; Week 8 L2).

Why we need it

Why we need them: programs exchange **objects** and make **calls**; networks move only bytes. These layers bridge the two, at a cost we can measure.

Source: Birrell & Nelson, ACM TOCS 1984; gRPC docs; PyTorch torch.distributed.rpc docs; mpi4py docs.

Learning goals for today

- **Serialize** an array and a list, and predict the bytes each format produces.
- **Explain** what an RPC stub does, and what can go wrong when the network fails.
- **Compare** Ethernet, InfiniBand, and RDMA by what they do to α and to β.
- **Describe** how a datacenter connects thousands of servers (leaf–spine).

A puzzle to start

Raw TCP round trip
(Week 6 L1): $\approx$ 48 µs

XML-RPC: add(2, 3)
same laptop, a few hundred bytes

How long does one remote add(2, 3) take?

Answer

$\approx$ **358 µs**: 7.5$\times$ the raw round trip, for almost no bytes. The layers **above** the socket cost more than the network.

Source: our laptop, made for these slides: Apple M1 Pro, macOS, Python 3.14, NumPy 2.5, loopback; median of 5 runs.

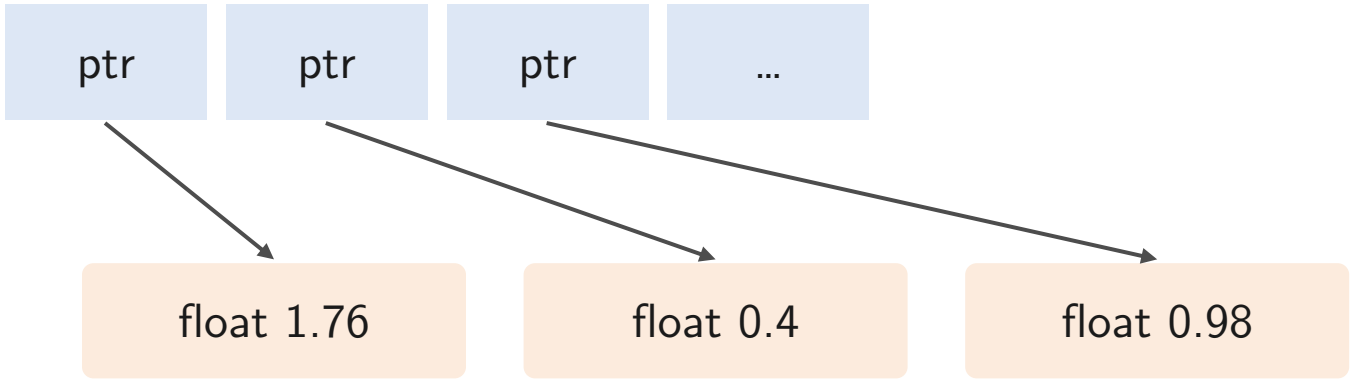
Part 1

Serialization

How does an object become bytes, and what does it cost?

Objects are full of pointers

A Python list:



8-byte pointers to 24-byte objects

A NumPy array:

dtype, shape	1.76	0.40	0.98	2.24	1.87	...

Our gradients

List of 34,338 floats: $\approx$ **1.1 MB**, scattered

Array: **137 KB**, contiguous

Watch out

Addresses mean nothing elsewhere: serialization writes the **values**.

Source: Python documentation, sys.getsizeof (CPython 3.14); NumPy documentation, ndarray; measured.

One array, four formats

Format	Bytes per number	Total for 34,338
raw float32 (`tobytes`)	4	137 KB
pickle of a list	9	309 KB
JSON text	$\approx$ 21	709 KB
XML-RPC text	$\approx$ 52	1.77 MB

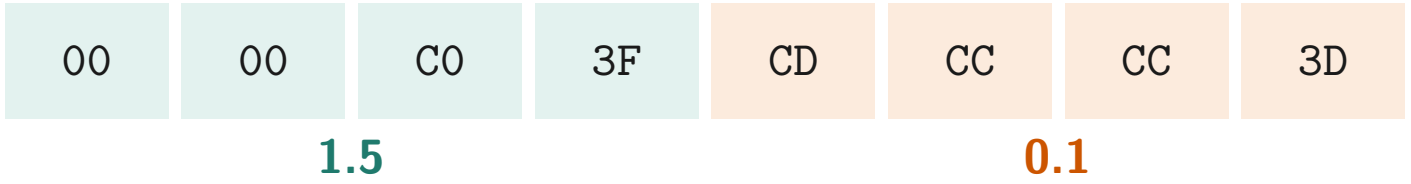
Key idea

Binary is **small**; text is **readable** and portable. The same numbers differ **13×** in size.

Source: our laptop, made for these slides: Apple M1 Pro, macOS, Python 3.14, NumPy 2.5, loopback; median of 5 runs.

A tensor as bytes

```
g = np.array([1.5, 0.1], dtype=np.float32)
b = g.tobytes()              # 8 bytes, no header
b.hex()                      # '0000c03fcdcccc3d'
h = np.frombuffer(b, dtype=np.float32)
```



Send first

dtype: float32, 4 bytes

shape: (34,338,)

byte order: little-endian

then the raw buffer

Key idea

A tensor is **header + raw buffer**. Send the header first so the receiver can read the rest.

Source: IEEE 754-2019 (binary32); NumPy documentation: tobytes, frombuffer, dtype byte order.

Serialization costs time, too



Sending 137 KB over our loopback TCP: $\approx$ **43 µs**

Key idea

Format and copy costs add to **every byte**, on both sides. Send arrays as **raw buffers**.

Source: our laptop, made for these slides: Apple M1 Pro, macOS, Python 3.14, NumPy 2.5, loopback; median of 5 runs.

Check yourself

Two minutes with a neighbour.

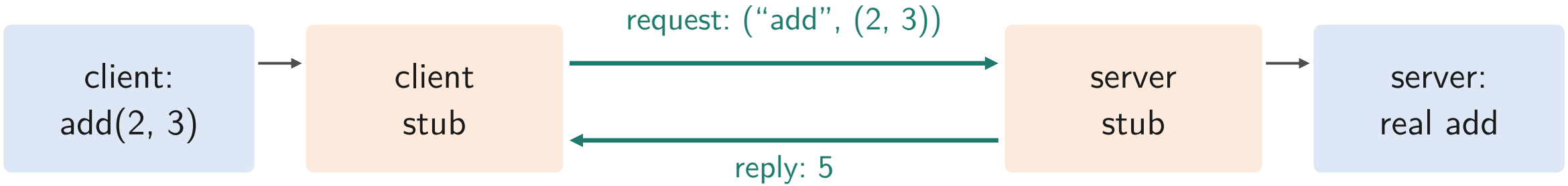
- Why can't a program send its Python list's memory as it is?
- The gradients become float16. How many raw bytes now? And as JSON?
- What must the receiver know before `np.frombuffer` can read the bytes?

Part 2

Remote procedure calls

How do we call a function on another machine, and what can go wrong?

A remote call that looks local



- **Stub**: code that serializes the call, sends it, and waits.
- The caller **blocks** until the reply comes back, like a local call.

Key idea

RPC = **serialize** + **request** + **run** + **serialize** + **reply**, hidden behind a function call.

Source: Birrell & Nelson, “Implementing Remote Procedure Calls,” ACM TOCS 2(1), 1984.

The cost of one call

$$\mathcal{T}(\text{call}) \approx 2\alpha + \beta \cdot (\text{bytes both ways}) + \mathcal{T}(\text{serialize}) + \mathcal{T}(\text{compute})$$

add(2, 3): almost no bytes $\Rightarrow$ T(call) $\approx 2\alpha \approx$ **48 µs** on our laptop

Key idea

Every call pays **2α** at least. Many tiny calls are latency-bound: **batch** them.

Source: Model: Week 6 L1 (Hockney 1994); our laptop, made for these slides: Apple M1 Pro, macOS, Python 3.14, NumPy 2.5, loopback; median of 5 runs.

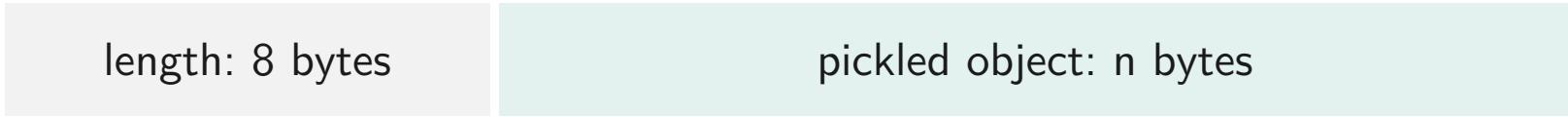
An RPC in a dozen lines: messages

```
def send_msg(s, obj):
    data = pickle.dumps(obj)
    s.sendall(len(data).to_bytes(8, "big") + data)

def recv_msg(s):
    n = int.from_bytes(recv_exact(s, 8), "big")
    return pickle.loads(recv_exact(s, n))
```

Watch out

`pickle.loads` can run **any code** the sender chose. Only between processes **you** trust.



Key idea

Framing: a length prefix tells the receiver where each message ends.

Source: Python documentation: pickle (security warning), socket.

An RPC in a dozen lines: stub and server

```python
# server
FUNCS = {"add": lambda a, b: a + b}
conn, _ = srv.accept()
while True:
    name, args = recv_msg(conn)
    send_msg(conn, FUNCS[name](*args))
```

```python
# client stub
def call(name, *args):
    send_msg(s, (name, args))
    return recv_msg(s)

call("add", 2, 3)       # 5
```

Our laptop: **61 µs** per call · gradients as an array: **110 µs** · as a list: **2.1 ms**

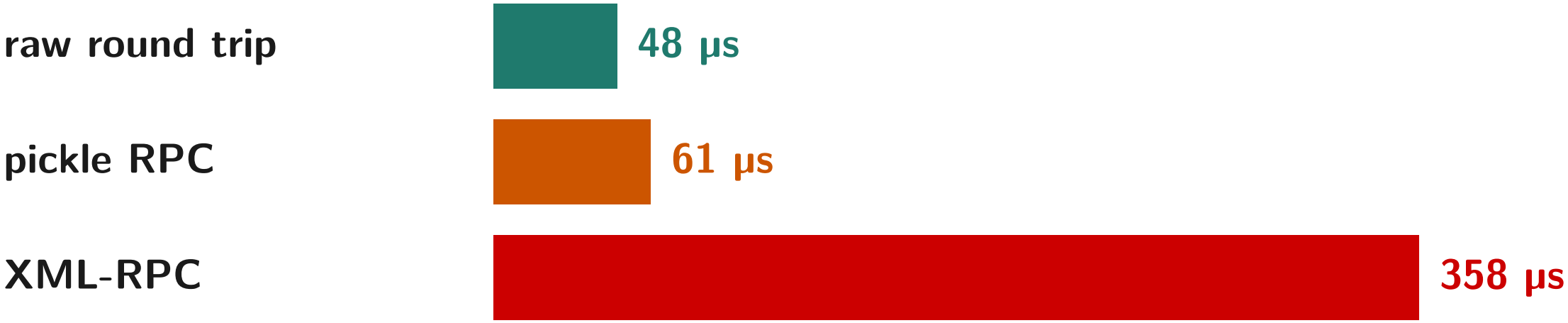
Key idea

A **stub** is just serialize, send, wait, deserialize. Libraries add errors, timeouts, and IDs.

Source: our laptop, made for these slides: Apple M1 Pro, macOS, Python 3.14, NumPy 2.5, loopback; median of 5 runs.

Where the time goes

add(2, 3) on our laptop, per call

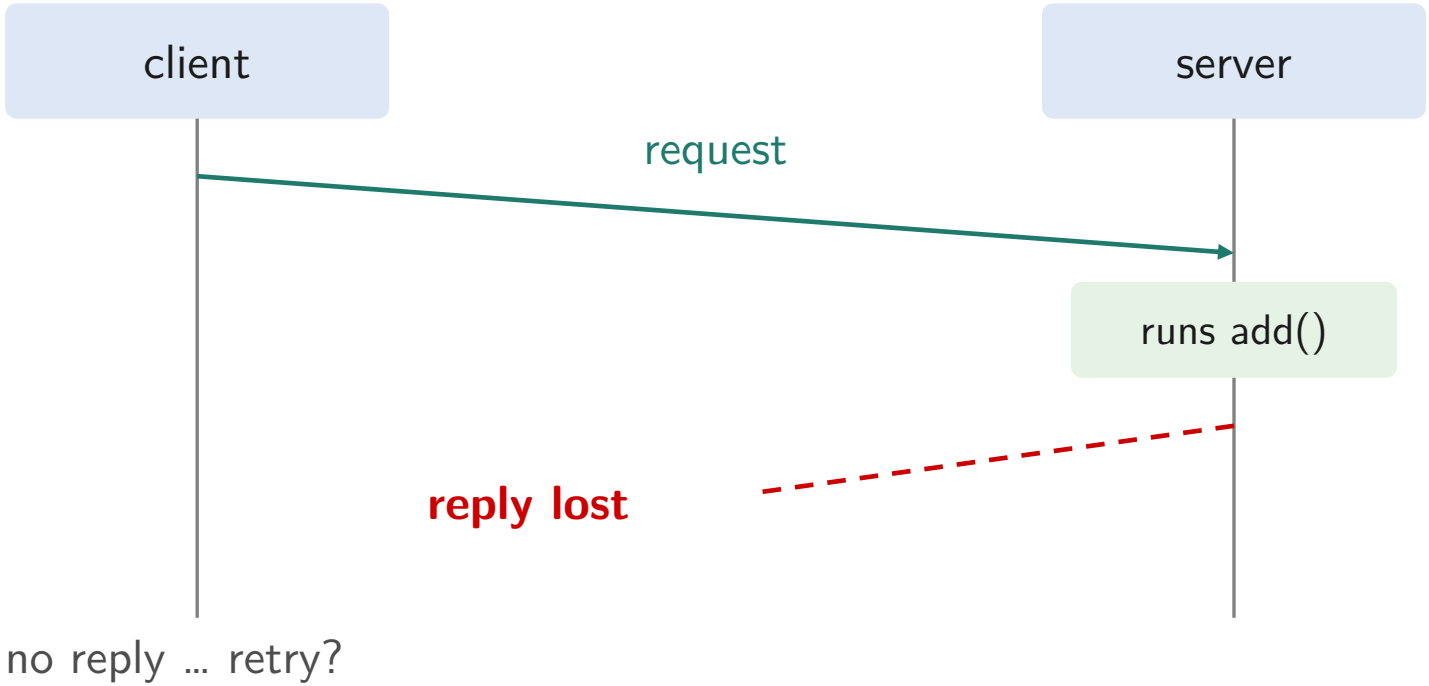


Watch out

Same network, same bytes: the **layers above the socket** set the cost. XML-RPC's α is 7.5× larger.

Source: our laptop, made for these slides: Apple M1 Pro, macOS, Python 3.14, NumPy 2.5, loopback; median of 5 runs.

When the network fails



- **At-least-once**: retry on timeout; may run **twice**. Safe only if repeating is harmless.
- **At-most-once**: request **IDs**; the server skips repeats.
- **Exactly-once**: not possible in general.

Watch out

No reply looks the same whether the request, the server, or the reply failed. Retries are safe only for **idempotent** calls.

Source: Birrell & Nelson, ACM TOCS 1984; Tanenbaum & van Steen, Distributed Systems, ch. 8 (RPC failures).

Check yourself

Two minutes with a neighbour.

- $\alpha = 25$ µs. How many tiny RPCs per second can one client make, one at a time? How would you send 10,000 values faster?
- Which is safe to retry: "set w to this value" or "subtract $\eta \cdot g$ from w"?
- Why must every RPC library have a **timeout**?

Part 3

Datacenter networks

How do thousands of servers talk fast enough to train one model?

Ethernet keeps getting faster

Line rate	IEEE standard	Year
1 Gb/s	802.3ab	1999
10 Gb/s	802.3ae	2002
100 Gb/s	802.3ba	2010
400 Gb/s	802.3bs	2017

In $\alpha + \beta n$

Each step: β (time per byte) $\div$ **10** or more

400 Gb/s $=$ **50 GB/s**

Question

Faster Ethernet shrinks β. Does it shrink α?

Source: IEEE Std 802.3ab-1999, 802.3ae-2002, 802.3ba-2010, 802.3bs-2017, 802.3df-2024.

A fast wire is not enough

one 4 KB message at 400 Gb/s



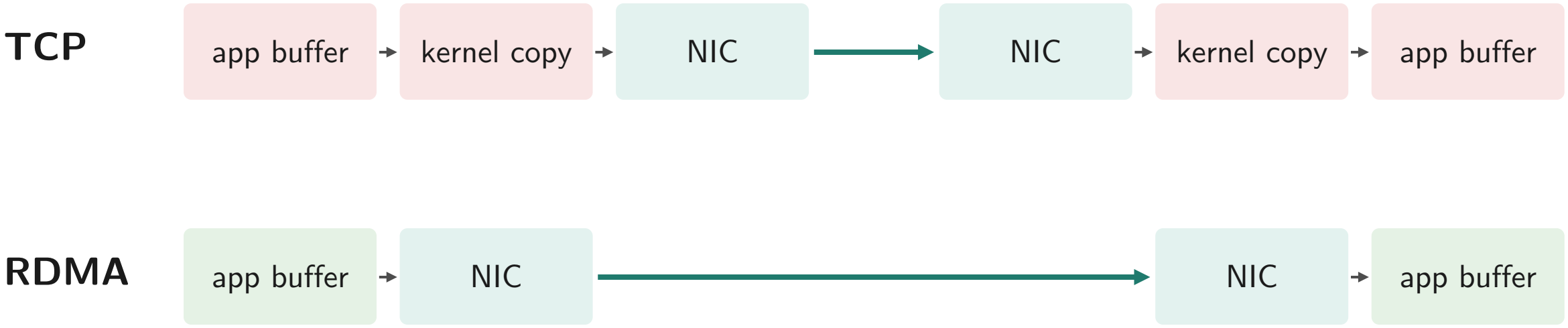
system calls · copies into the kernel · protocol in software · wake the receiver

Watch out

In a datacenter, α **is software**: $\approx 291\times$ the wire time for this message.

Source: IEEE 802.3bs; our laptop (Week 6 L1); Kalia, Kaminsky & Andersen, "Datacenter RPCs can be General and Fast," NSDI 2019 (2.3 µs median RPC).

RDMA: the network card does the copying



kernel bypass · **zero copy** · remote CPU not involved

Key idea

RDMA cuts α from tens of microseconds to a few: the NIC moves bytes between registered buffers directly.

Source: InfiniBand Architecture Specification (IBTA); Kalia, Kaminsky & Andersen, "Datacenter RPCs can be General and Fast," NSDI 2019 (2.3 µs median RPC).

InfiniBand and RoCE

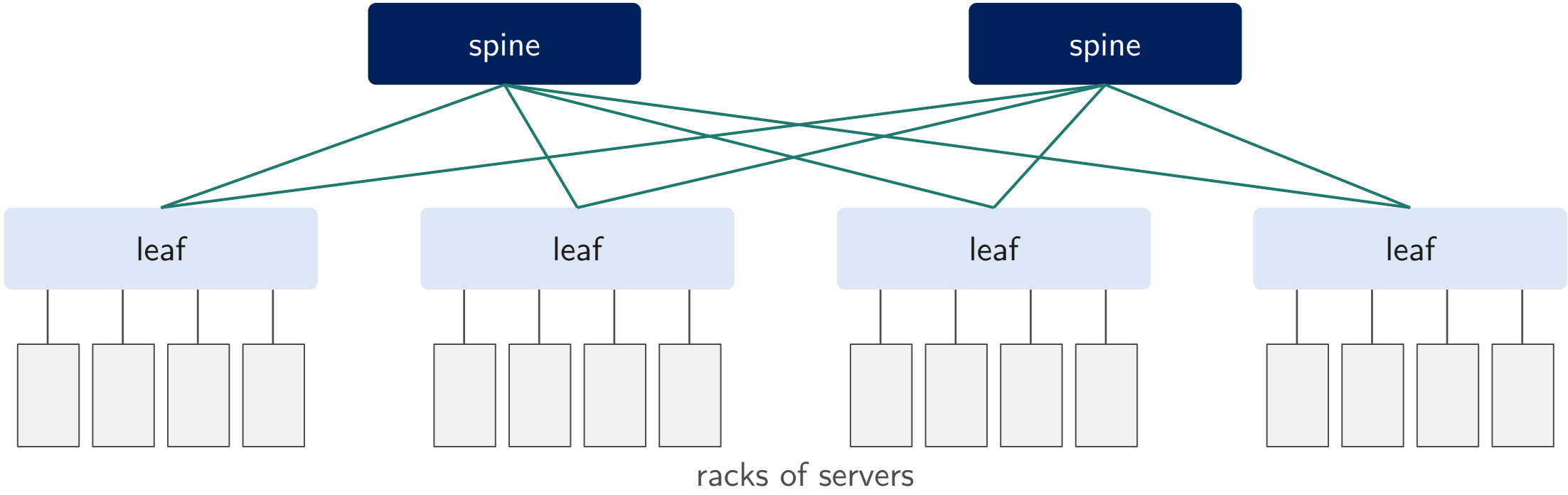
	InfiniBand	**RoCE v2**
Network	its own cards and switches	ordinary Ethernet
Losses	none: credit-based flow	switches must be tuned
Typical port	NDR **400 Gb/s**	100–400 Gb/s

Key idea

Both give **RDMA**: a small α and a line-rate β. InfiniBand is built for it; RoCE adds it to Ethernet.

Source: IBTA InfiniBand Architecture Specification; NVIDIA ConnectX-7 brief; IBTA RoCE v2 (Annex A17).

Racks and a leaf–spine network



Key idea

Every leaf links to every spine: any two servers are **3 switch hops** apart. 64-port switches reach **65,536** servers.

Source: Al-Fares, Loukissas & Vahdat, "A Scalable, Commodity Data Center Network Architecture," SIGCOMM 2008; NVIDIA Quantum-2 QM9700 datasheet.

Oversubscription

up: 6×100 Gb/s $=$ **600 Gb/s**

leaf switch

down: 48×25 Gb/s $=$ **1,200 Gb/s**

$$1{,}200 : 600 \;=\; 2 : 1$$

all 48 send out at once:
each gets **12.5 Gb/s**, not 25

Watch out

An all-reduce makes **every** server send at once. Training fabrics are built **1:1** (non-blocking).

Source: Al-Fares et al., SIGCOMM 2008; Singh et al., SIGCOMM 2015; switch configuration illustrative.

Check yourself

Two minutes with a neighbour.

- For many 1 KB messages, which helps more: RDMA or a $4\times$ faster link? Why?
- In a leaf–spine network, how many switches lie between servers in different racks?
- A leaf has 32 down and 16 up ports, all 400 Gb/s. Ratio? Effect on an all-reduce?

IN-CLASS BUILD

Objects, calls, and networks

Pairs: predict, then measure or compute.

Exercise 1: turn objects into messages

```
g = np.random.randn(34_338).astype(np.float32)
xs = g.tolist()                    # Python floats
out = {"raw": g.tobytes(),
       "pickle": pickle.dumps(xs),
       "json": json.dumps(xs).encode(),
       "xml": xmlrpc.client.dumps((xs,))}
for k, b in out.items():
    print(k, len(b), len(b) / len(xs))
```

DO

1. **Predict** bytes per number for each format.
2. Run it. Which surprised you?

STRETCH

Time each format, and the way back (`frombuffer`, `loads`).

Source: Python 3 documentation: pickle, json, xmlrpc.client, xmlrpc.server; NumPy: ndarray.tobytes, frombuffer.

Exercise 2: call a function on another machine

```
# server.py
srv = SimpleXMLRPCServer(("", 8000))
srv.register_function(lambda a, b: a + b, "add")
srv.register_function(sum, "total")
srv.serve_forever()
# client.py
p = ServerProxy(f"http://{HOST}:8000")
print(p.add(2, 3))              # runs on the server
t0 = time.perf_counter(); p.total(xs)
print(time.perf_counter() - t0)
```

DO

1. **Predict** both calls with $\alpha + \beta n$.
2. Measure them.
3. Where did the time go?

STRETCH

Time the pickle RPC from Part 2 on the same calls.

Source: Python documentation: xmlrpc.server, xmlrpc.client.

Exercise 3: compare interconnects

Network	1 KB	137 KB	4 GB
1 Gb/s Ethernet, TCP			
100 Gb/s Ethernet, TCP			
400 Gb/s InfiniBand, RDMA			

USE

TCP: $\alpha = 25$ µs · RDMA: $\alpha = 2$ µs

B = line rate ÷ 8; KB = 10^3 bytes

STRETCH

$n_{1/2} = \alpha \cdot B$ for each. Which cells are latency-bound?

Source: α assumed (our laptop; Kalia et al., NSDI 2019); B: IEEE 802.3, InfiniBand NDR line rates.

Solutions: formats and calls

- Bytes per number: **4** raw, **9** pickle, $\approx$ **21** JSON, $\approx$ **52** XML.
- XML-RPC add: **358 µs**, predicted $\approx$ 48 µs: a new connection, HTTP, XML.
- XML-RPC total(xs): **83 ms**, but $\alpha + \beta n$ predicts $\approx$ **0.3 ms**.

Check

When the measurement is far above $\alpha + \beta n$, the gap is **software**: format, copies, connections.

Source: our laptop, made for these slides: Apple M1 Pro, macOS, Python 3.14, NumPy 2.5, loopback; median of 5 runs.

Solutions: three interconnects

Network	1 KB	137 KB	4 GB
1 Gb/s Ethernet, TCP	33 µs	1.1 ms	32 s
100 Gb/s Ethernet, TCP	25 µs	36 µs	320 ms
400 Gb/s InfiniBand, RDMA	2 µs	4.7 µs	80 ms

Key idea

Small messages: α decides (RDMA wins). Large messages: **B** decides (the fastest link wins).

Source: Model estimates; α assumed (our laptop; Kalia et al., NSDI 2019); IEEE 802.3; InfiniBand NDR.

Key takeaways

- **Serialization** follows pointers and writes values; binary is small, text is portable.
- Send arrays as **header + raw buffer**; never unpickle untrusted bytes.
- **RPC**: a stub hides two messages behind a call; every call pays $\geq \mathbf{2}\alpha$.
- No reply is ambiguous: retries are safe only for **idempotent** calls.
- **RDMA** (InfiniBand, RoCE) cuts α; faster links cut β; training fabrics are **1:1**.

Next time, and readings

When	What
Week 7, Lecture 1	Message passing with mpi4py; fit α and β
This week	D3 checkpoint (D3 due Week 7)
Week 7, Lecture 2	Collectives; T2 and Lab 4 released after it

READINGS

Birrell & Nelson, "Implementing Remote Procedure Calls," ACM TOCS 1984.

Al-Fares et al., "A Scalable, Commodity Data Center Network Architecture," SIGCOMM 2008 (§1–3).